EXPERIMENT NO. 5

Registration form using Regular Expressions

AIM: To validate the fields of a registration form using Regular Expressions (RegEx) in JavaScript to ensure that user inputs such as name, email, password, and phone number follow the required format.

THEORY:

Form Validation in Web Development

Form validation is the process of checking whether the user input entered into a form is correct, complete, and in the required format before submitting the data to the server. Validation improves data accuracy and prevents invalid information from being stored in the system.

Validation can be performed using **HTML attributes**, **JavaScript**, or **server-side programming**. In this experiment, validation is implemented using **JavaScript with Regular Expressions**.

Regular Expressions (RegEx)

A Regular Expression is a pattern used to match character combinations in strings. It is widely used in JavaScript for searching, validating, and manipulating text.

Regular expressions allow developers to check whether user input matches a specific pattern.

Some common uses of RegEx include:

- Validating email addresses
- Checking password strength
- Verifying phone numbers
- Restricting characters in names

For example:

- ^[A-Za-z]+$ → Allows only alphabets and spaces
- ^\d{10}$ → Validates a 10-digit phone number
- ^[^\s@]+@[^\s@]+\.[^\s@]+$ → Validates email format

JavaScript Validation

JavaScript can capture form submission events and validate inputs before the form is submitted. If any field does not match the required pattern, an error message is displayed and the form submission is prevented.

In this experiment, JavaScript checks the following fields:

- **Name** – must contain only alphabets
- **Email** – must follow a valid email format
- **Phone Number** – must contain exactly 10 digits
- **Password** – must be at least 6 characters long

If the entered values match the regular expression patterns, the form is successfully validated.

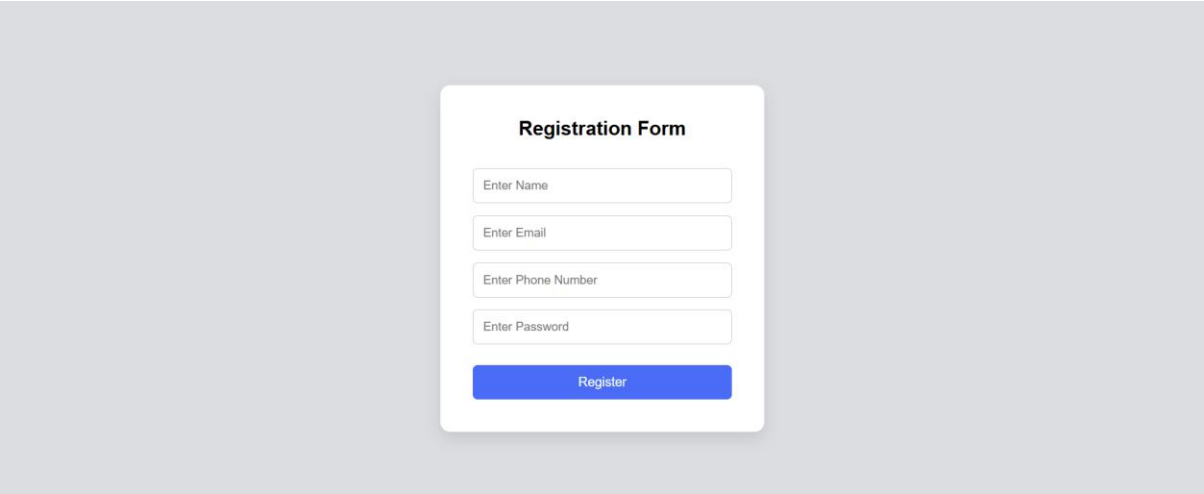
CODE:

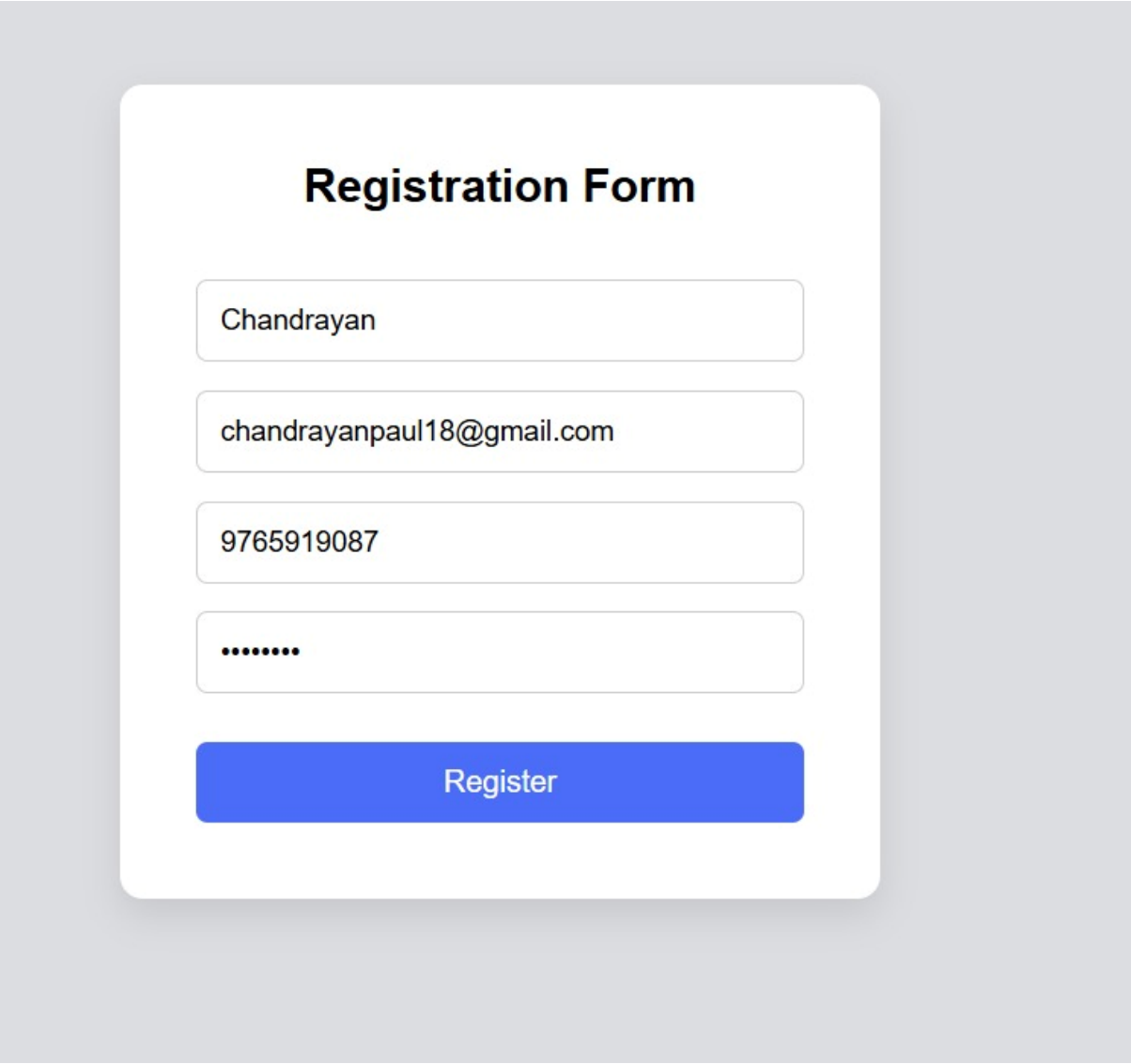
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Registration Form</title>

<style>

*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial, sans-serif;
}

body{
background:#dcdde1;
height:100vh;
display:flex;
justify-content:center;
align-items:center;
}

.container{
background:white;
padding:40px;
border-radius:12px;
width:400px;
box-shadow:0 8px 20px rgba(0,0,0,0.1);
}

h2{
text-align:center;
margin-bottom:25px;
}

input{
width:100%;
padding:12px;
margin-top:10px;
border:1px solid #ccc;
border-radius:6px;
font-size:15px;
}

button{
width:100%;
margin-top:20px;
padding:12px;
background:#4a6cf7;
border:none;
```

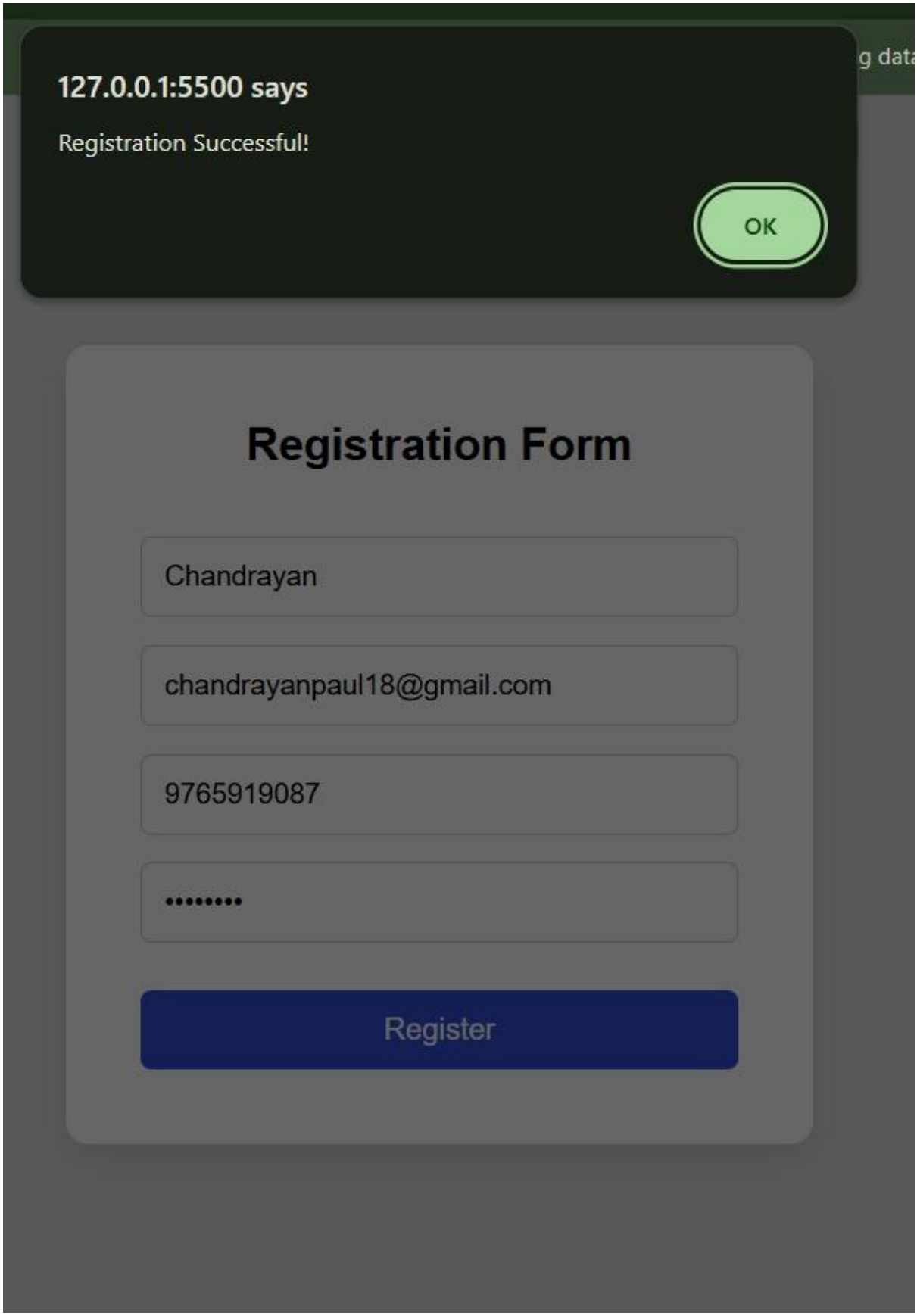
```
color:white;
font-size:16px;
border-radius:6px;
cursor:pointer;
}

button:hover{
background:#3651d4;
}

.error{
color:red;
font-size:14px;
margin-top:5px;
}

</style>

</head>

<body>

<div class="container">

<h2>Registration Form</h2>

<form id="registerForm">

<input type="text" id="name" placeholder="Enter Name">
<div class="error" id="nameError"></div>

<input type="text" id="email" placeholder="Enter Email">
<div class="error" id="emailError"></div>

<input type="tel" id="phone" placeholder="Enter Phone Number">
<div class="error" id="phoneError"></div>

<input type="password" id="password" placeholder="Enter Password">
<div class="error" id="passwordError"></div>

<button type="submit">Register</button>

</form>

</div>

<script>

const form=document.getElementById("registerForm");

form.addEventListener("submit",function(e){

e.preventDefault();
```

```
let name=document.getElementById("name").value.trim();
let email=document.getElementById("email").value.trim();
let phone=document.getElementById("phone").value.trim();
let password=document.getElementById("password").value.trim();

let namePattern=/^[A-Za-z ]+$/;
let emailPattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;
let phonePattern=/^[6-9]\d{9}$/;
let passwordPattern=/^.{6,}$/;

let valid=true;

document.getElementById("nameError").textContent="";
document.getElementById("emailError").textContent="";
document.getElementById("phoneError").textContent="";
document.getElementById("passwordError").textContent="";

if(!namePattern.test(name)){
document.getElementById("nameError").textContent="Enter valid name (letters
only)";
valid=false;
}

if(!emailPattern.test(email)){
document.getElementById("emailError").textContent="Enter valid email";
valid=false;
}

if(!phonePattern.test(phone)){
document.getElementById("phoneError").textContent="Enter valid 10 digit phone
number";
valid=false;
}

if(!passwordPattern.test(password)){
document.getElementById("passwordError").textContent="Password must be at
least 6 characters";
valid=false;
}

if(valid){
alert("Registration Successful!");
form.reset();
}

});

</script>

</body>
</html>
```

OUTPUT:


Registration Form
Enter Name
Enter Email
Enter Phone Number
Enter Password
Register


Registration Form
Chandrayan
chandrayanpaul18@gmail.com
9765919087
••••••••
Register



Validate login credentials without refreshing the page.(AJAX)

AIM: To validate login credentials without refreshing the webpage using **AJAX (Asynchronous JavaScript and XML)** and JavaScript.

THEORY:

AJAX (Asynchronous JavaScript and XML)

AJAX is a web development technique used to send and receive data from a server asynchronously without reloading the entire webpage. It allows webpages to update parts of the content dynamically, improving the speed and responsiveness of web applications.

AJAX works by sending requests from the browser to the server using JavaScript. The server processes the request and sends a response back to the browser. The browser then updates the webpage dynamically using the received data.

Login Validation Using AJAX

In this experiment, a login form is created using HTML. When the user enters the username and password and clicks the login button, JavaScript sends the entered credentials asynchronously to a server-side script.

The server checks whether the credentials are correct. If the credentials match the stored values, the server returns a success message; otherwise, it returns an error message.

The webpage then displays the result dynamically without reloading the page. This demonstrates how AJAX improves user experience by making web applications faster and more interactive.

CODE:

logincredentials.html

```
<!DOCTYPE html>
<html>
<head>
<title>AJAX Login</title>

<style>

body{
font-family:Arial;
background:#eef2ff;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}

.container{
background:white;
padding:35px;
border-radius:12px;
width:320px;
box-shadow:0 8px 20px rgba(0,0,0,0.1);
text-align:center;
}

h2{
margin-bottom:20px;
}
```

```
input{
width:100%;
padding:10px;
margin:10px 0;
border:1px solid #ccc;
border-radius:6px;
font-size:15px;
}

button{
width:100%;
padding:10px;
background:#4a6cf7;
border:none;
color:white;
font-size:16px;
border-radius:6px;
cursor:pointer;
}

button:hover{
background:#3249c5;
}

#result{
margin-top:12px;
font-weight:bold;
}

</style>
</head>

<body>

<div class="container">

<h2>Login</h2>

<input type="text" id="username" placeholder="Enter Username">
<input type="password" id="password" placeholder="Enter Password">

<button onclick="login()">Login</button>

<div id="result"></div>

</div>

<script>

function login(){

const username=document.getElementById("username").value;
const password=document.getElementById("password").value;
```

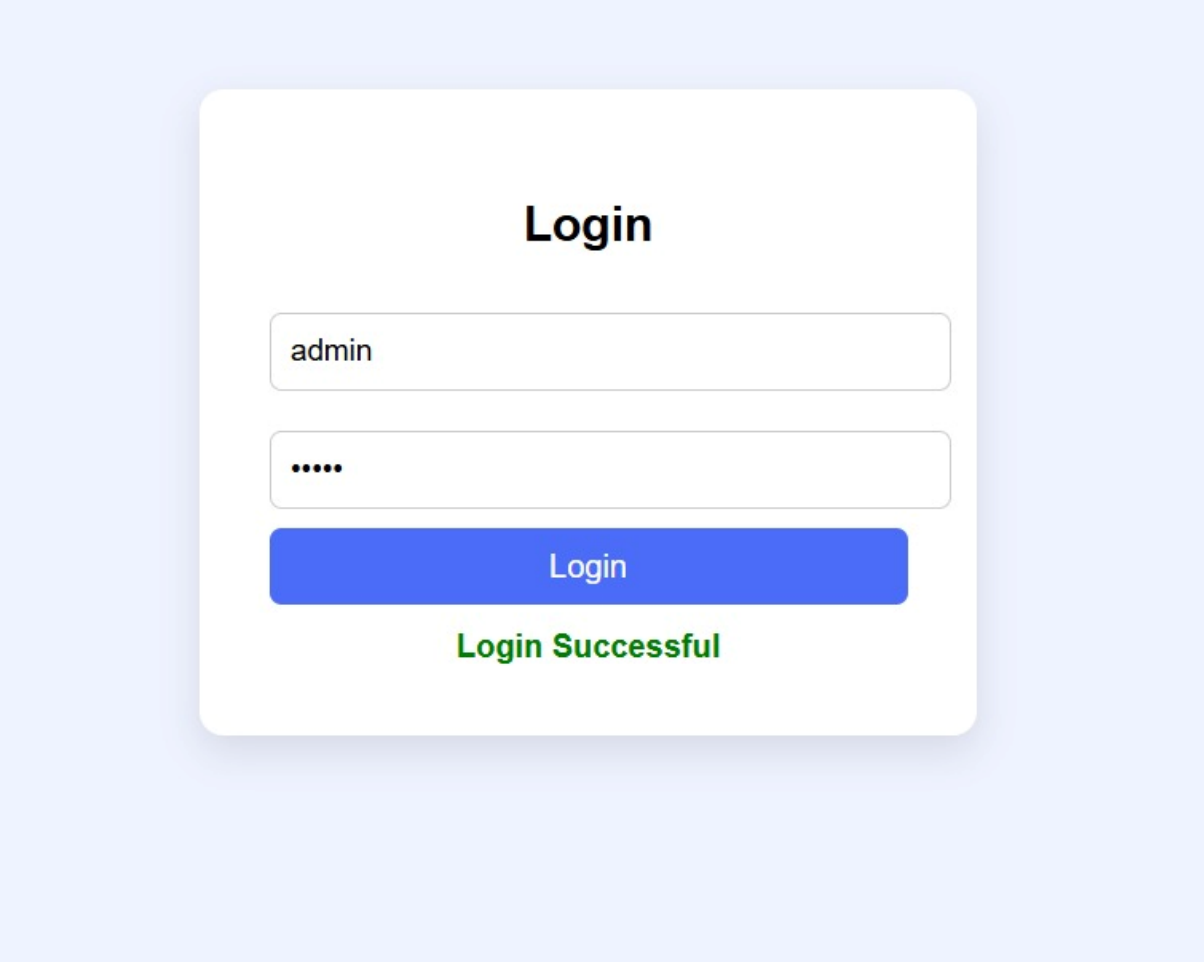
```
const xhr=new XMLHttpRequest();

xhr.open("GET","users.json",true);

xhr.onreadystatechange=function(){

if(xhr.readyState===4 && xhr.status===200){

const data=JSON.parse(xhr.responseText);

let valid=false;

data.users.forEach(function(user){

if(user.username===username && user.password===password){
valid=true;
}

});

const result=document.getElementById("result");

if(valid){
result.innerHTML="<span style='color:green'>Login Successful</span>";
}
else{
result.innerHTML="<span style='color:red'>Invalid Username or
Password</span>";
}

}

};

xhr.send();

}

</script>

</body>
</html>
```

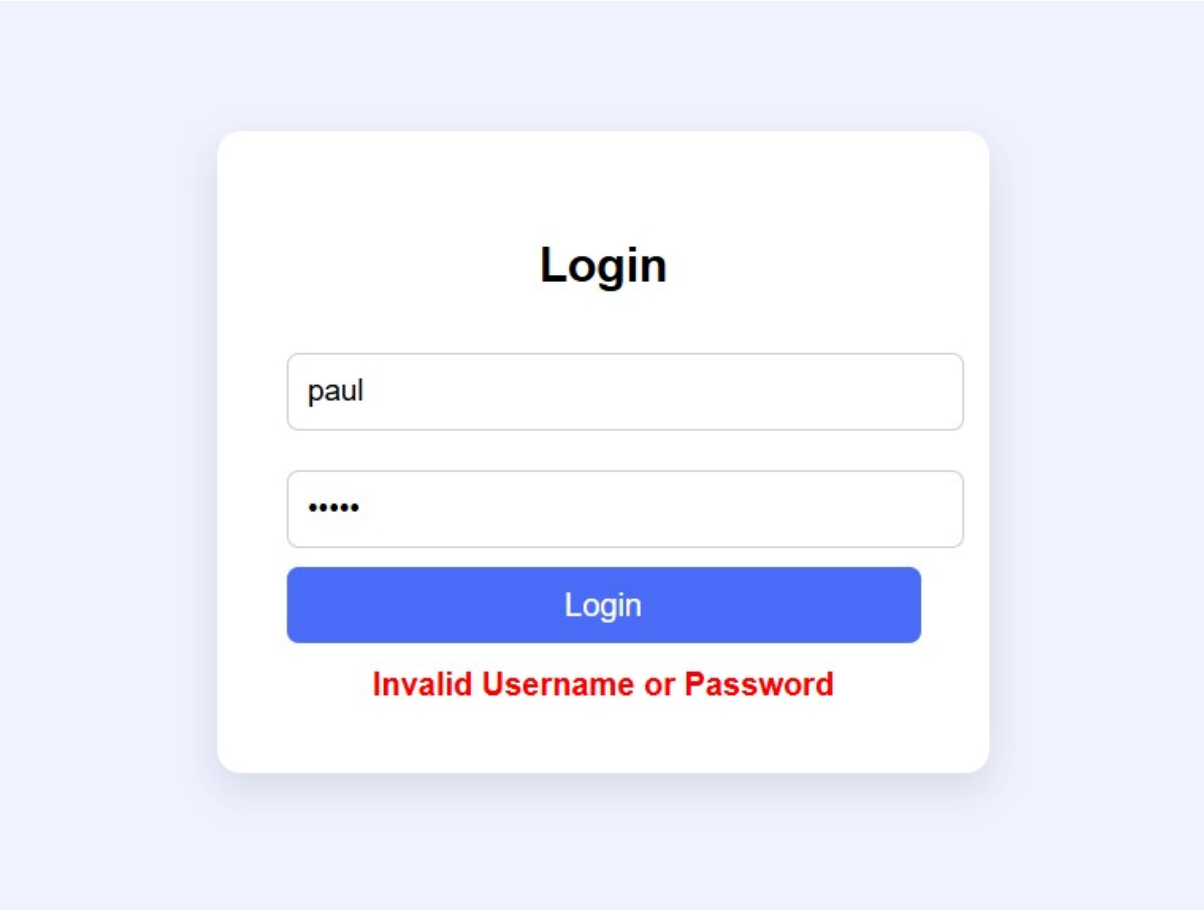
users.json

```
{
  "users":[
    {
      "username":"admin",
      "password":"12345"
    },
    {
      "username":"user",
      "password":"pass123"
```

```
    }
  ]
}
```

OUTPUT:





Read contents of the file data.json. Using AJAX

AIM: To read and display the contents of a JSON file (data.json) using AJAX (Asynchronous JavaScript and XML) without refreshing the webpage

THEORY:

AJAX (Asynchronous JavaScript and XML) is a web development technique used to send and receive data asynchronously between a client and a server without refreshing the entire webpage. It improves the performance and responsiveness of web applications by updating only the required parts of a webpage.

AJAX commonly uses the **XMLHttpRequest object** or modern **Fetch API** to request data from the server.

JSON (JavaScript Object Notation) is a lightweight data format used to store and exchange data. It is easy to read and write and is widely used for transmitting data between a server and a web application.

In this experiment, a JSON file named **data.json** stores sample data such as names and ages. JavaScript uses AJAX to send a request to read the JSON file. Once the data is received, it is converted into a JavaScript object using

JSON.parse() and displayed dynamically on the webpage without reloading the page.

CODE:

readcontent.html

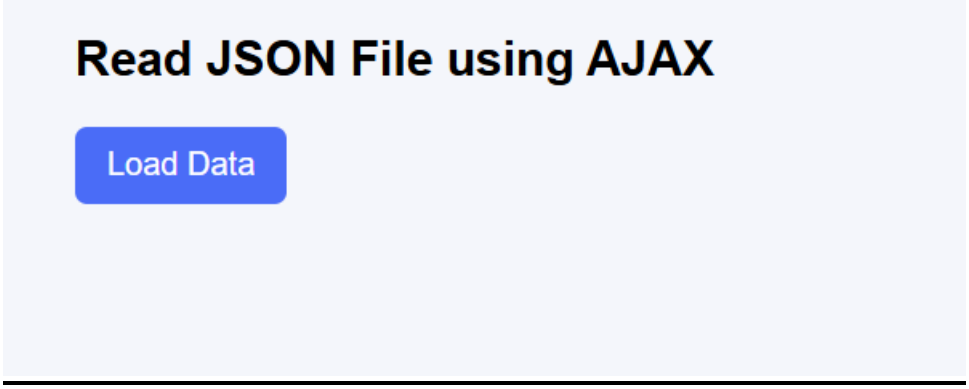
```
<!DOCTYPE html>
<html>
<head>
<title>Read JSON using AJAX</title>

<style>

body{
font-family:Arial;
margin:40px;
background:#f4f6fb;
}

button{
padding:10px 16px;
background:#4a6cf7;
border:none;
color:white;
font-size:16px;
border-radius:6px;
cursor:pointer;
}

button:hover{
background:#3249c5;
}

#output{
margin-top:20px;
font-size:18px;
}

</style>

</head>

<body>

<h2>Read JSON File using AJAX</h2>

<button onclick="loadData()">Load Data</button>

<div id="output"></div>

<script>
```

```
function loadData(){

var xhr=new XMLHttpRequest();

xhr.open("GET","data.json",true);

xhr.onreadystatechange=function(){

if(xhr.readyState==4 && xhr.status==200){

var data=JSON.parse(xhr.responseText);

var result="";

data.students.forEach(function(student){

result+="Name: "+student.name+" | Age: "+student.age+"<br>";

});

document.getElementById("output").innerHTML=result;

}

};

xhr.send();

}

</script>

</body>
</html>
```

data.json

```
{
  "students":[
    {
      "name":"Rahul",
      "age":20
    },
    {
      "name":"Priya",
      "age":21
    },
    {
      "name":"Amit",
      "age":19
    }
  ]
}
```

OUTPUT:



CONCLUSION: This experiment demonstrated how JavaScript can be used to validate form inputs using regular expressions and how AJAX enables asynchronous data exchange without refreshing the webpage. It also showed how JSON data can be retrieved and displayed dynamically, improving interactivity and user experience.